

Caption: A Comprehensive Measure of Well-Being

Project Description:

The Human Development Index (HDI) Predictor harnesses Machine Learning to estimate a country's level of human development, offering users a fast and interactive prediction experience. The platform uses four key indicators: Life Expectancy, Mean Years of Schooling, Expected Years of Schooling, and Gross National Income (GNI) per Capita to predict the HDI score and classify the result into Very High, High, Medium, or Low Human Development.

Utilizing a trained Linear Regression model, the HDI Predictor processes user inputs to generate meaningful predictions and insights. Built with Python, Scikit-learn, Pandas, Flask, HTML, CSS, and data visualization libraries, the platform provides a seamless and user-friendly experience. By combining health, education, and living-standard indicators, the HDI Predictor helps students, researchers, and policymakers better understand the factors that influence a country's overall human development.

Scenarios

Scenario 1: A user enters high values for Life Expectancy, Mean Years of Schooling, Expected Years of Schooling, and GNI per Capita. The machine learning model processes these values and predicts a high HDI score. The system classifies the result under the Very High Human Development category.

This scenario helps users understand how strong healthcare systems, better educational opportunities, and higher income levels contribute to improved human development.

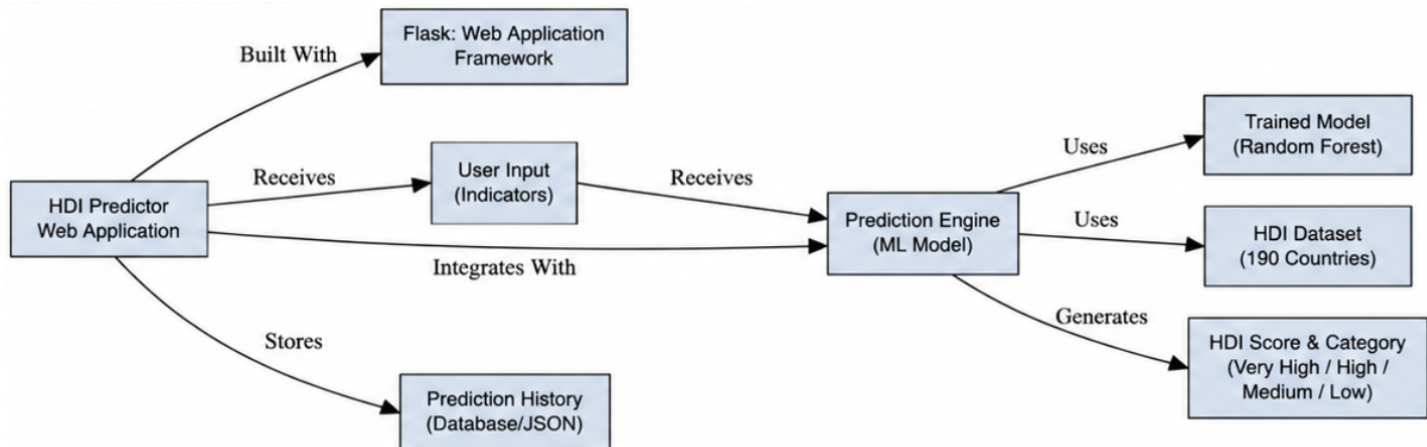
Scenario 2: A policymaker or researcher enters moderate values for life expectancy, educational attainment, expected schooling, and income. The model predicts a Medium HDI score based on the provided indicators.

The result can help users understand the areas in which improvements are required. For example, improvements in healthcare, educational opportunities, or income generation may contribute to a higher level of human development.

Scenario 3 : A researcher enters relatively low values for Life Expectancy, Mean Years of Schooling, Expected Years of Schooling, and GNI per Capita. The system predicts a Low HDI score and classifies the country under the Low Human Development category.

This prediction can help identify significant development challenges and highlight the need for improvements in healthcare, education, and economic opportunities.

Technical Architecture:



Description: The HDI Predictor follows a modular architecture consisting of a presentation layer, application layer, machine learning layer, and data storage layer.

The frontend is developed using HTML and CSS and provides a user-friendly interface through which users can enter the four required development indicators. The Flask backend receives the input values, validates and processes them, and sends the prepared values to the trained machine learning model.

The machine learning layer contains the trained Linear Regression model and the preprocessing components required to generate the HDI prediction. The predicted score is then converted into an appropriate human development category and displayed to the user.

The application also uses an SQLite database file for local data storage. The overall architecture ensures that the frontend, backend, machine learning model, and data storage components work together efficiently.

(Note: If you are using database in your project definitely you need to add ER Diagram along with description)

• Pre-requisites:

- Flask Framework Knowledge: Flask Documentation
- Machine Learning Familiarity: Scikit-learn Documentation
- Data Analysis Skills: Pandas and NumPy Documentation
- Data Visualization Knowledge: Matplotlib and Seaborn Documentation
- HTML and CSS Skills: W3Schools HTML/CSS Tutorials
- Python Programming Proficiency: Python Documentation
- Development Environment Setup: Visual Studio Code and Python Installation Guide
- SQLite Database Familiarity: SQLite Documentation

Project Workflow:

Milestone 1: Environment Setup and Package Installation

- **Activity 1.1:** Download and install all required Python packages, machine learning libraries, and Flask dependencies.
- **Activity 1.2:** Create and organize the complete project folder structure.

Milestone 2: Importing Required Libraries

- **Activity 2.1:** Import NumPy, Pandas, Matplotlib, Seaborn, Scikit-learn, Pickle, Flask, and other required modules.
-

Milestone 3: Dataset Download and Understanding

- **Activity 3.1:** Obtain and load the HDI dataset.
- **Activity 3.2:** Understand the dataset structure, features, target variable, and data types.
- **Activity 3.3:** Perform exploratory data analysis and data visualization.

Milestone 4: Data Preprocessing and Label Encoding

- Activity 4.1: Select dependent and independent variables.
- Activity 4.2: Check and handle missing values.
- Activity 4.3: Encode categorical values where required.
- Activity 4.4: Prepare the processed dataset for model development.

Milestone 5: Dividing the Dataset into Train and Test Data

- **Activity 5.1:** Split the processed dataset into training and testing sets.

Milestone 6: Fitting the Model

- **Activity 6.1:** Train the Linear Regression model.
- **Activity 6.2:** Generate predictions.
- **Activity 6.3:** Evaluate the model using regression metrics and visualizations.

Milestone 7: Saving the Model

- **Activity 7.1:** Save the trained model.
- **Activity 7.2:** Store the required model files for future predictions.

Milestone 8: Building the Flask Web Application

- **Activity 8.1:** Develop the Flask backend.
- **Activity 8.2:** Create and integrate HTML templates.
- **Activity 8.3:** Run, test, and validate the complete application.

Milestone 1: Environment Setup and Package Installation

Milestone 1 focuses on preparing the development environment required to build the HDI Predictor. This includes checking the Python installation, creating the project structure, setting up a virtual environment, and installing all required packages.

Activity 1.1: Download and Install Required Packages

Before the application can communicate with the Groq LLM, you need a valid Groq API key. Follow the steps below to create one and connect it securely to the project.

- **Step 1: Verify Python Installation** :Open the terminal in Visual Studio Code and run:

```
python --version
```

The development environment used for this project contains:

Python 3.13.9

- **Step 2: Create and Activate the Virtual Environment:**

Create the virtual environment using:

```
python -m venv venv
```

Activate it using:

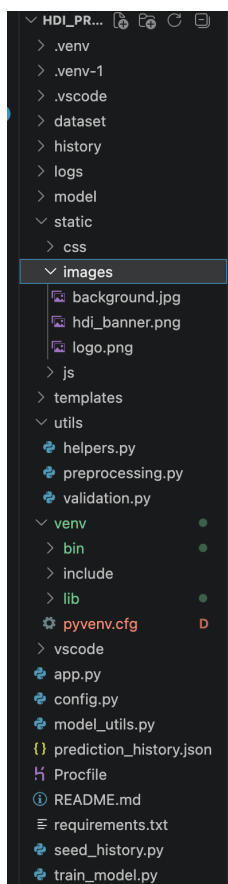
```
.\venv\Scripts\Activate.ps1
```

- **Step 3: Install Required Dependencies**

Install all project dependencies using:

pip install -r requirements.txt

The required libraries support dataset processing, visualization, machine learning, model saving, database operations, and Flask web development.



Milestone 2: Importing Required Libraries

Activity 2.1: Import Required Libraries

In this milestone, all the necessary Python libraries and modules required for data analysis, visualization, machine learning, model development, model evaluation, database operations, and web application creation are imported.

The following libraries are used in the project:

- NumPy: Used for numerical calculations and array operations.
- Pandas: Used for loading, exploring, cleaning, and processing the HDI dataset.
- Matplotlib: Used for creating graphs and visualizing data patterns.
- Seaborn: Used for creating statistical visualizations and correlation heatmaps.
- Scikit-learn: Used for splitting the dataset, preprocessing data, training the Linear Regression model, generating predictions, and evaluating model performance.
- Pickle: Used for saving and loading the trained machine learning model and preprocessing components.
- Flask: Used for developing the web application backend, handling user requests, processing inputs, and displaying prediction results.
- SQLite: Used for local database storage and managing application data.

These
all the

for
complete

Index
dataset
model
prediction
application

**Milestone
3: Dataset
Download
and**

```

app.py
app.py > ...
1  """
2  HDI Predictor Flask Application
3  """
4
5  import webbrowser
6  from threading import Timer
7  from os import environ
8
9  from flask import Flask, render_template, request
10
11  from config import SECRET_KEY
12  from model_utils import predict_hdi
13  from seed_history import (
14      save_prediction,
15      load_history,
16      initialize_history
17  )
18
19  app = Flask(__name__)
20  app.secret_key = SECRET_KEY
21
22  # Create history file if it doesn't exist
23  initialize_history()
24
25
26  def open_browser():
27      webbrowser.open_new("http://127.0.0.1:5001")
28
29
30  @app.route("/")
31  def home():
32      return render_template("index.html")
33
34
35  @app.route("/predict", methods=["POST"])
36  def predict():
37
38      try:
39          country = request.form["country"]
40
41          life_expectancy = float(request.form["life_expectancy"])

```

libraries provide
required
functionalities
developing the
Human
Development
Predictor, from
analysis and
training to
and web
integration.

Understanding Activity 3.1: Download and Load the Dataset

In this activity, the Human Development Index dataset is obtained and stored inside the dataset folder of the project. The dataset contains information related to the major indicators used to measure human development.

The dataset is loaded into the development environment using the Pandas library. Loading the dataset allows the data to be explored, analyzed, cleaned, and prepared for machine learning.

The main indicators used in the project are:

- Life Expectancy
- Mean Years of Schooling
- Expected Years of Schooling
- GNI per Capita

The target variable used for prediction is the HDI Score.

The dataset provides the foundation for training the machine learning model and understanding the relationship between health, education, income, and human development.

EXPLORER

...

HDI_PREDICTION_FINAL_P...

.venv

bin

lib

.gitignore

.lock

CACHEDIR.TAG

pyvenv.cfg

.venv-1

.vscode

dataset

Human Development I...

history

logs

model

model_meta.json

model.pkl

static

templates

utils

helpers.py

preprocessing.py

validation.py

venv

bin

include

lib

pyvenv.cfg

vscode

app.py

config.py

model_utils.py

prediction_history.json

Procfile

README.md

requirements.txt

OUTLINE

TIMELINE

Human Development Index - Full.csv

X

dataset > Human Development Index - Full.csv > data

```
1 Country,Life expectancy,Expected years of schooling,Mean years of schooling,GNI per capita,HDI
2 Afghanistan,65.5,15.8,13.0,19108,0.634
3 Albania,84.8,16.0,3.4,45924,0.677
4 Algeria,68.5,12.4,14.9,30444,0.679
5 Andorra,84.4,19.7,14.6,10470,0.729
6 Angola,64.6,17.8,8.8,43948,0.688
7 Antigua and Barbuda,70.7,19.5,13.0,11340,0.66
8 Argentina,69.4,10.8,7.8,18593,0.527
9 Armenia,80.3,11.4,13.4,45908,0.748
10 Australia,80.3,9.9,13.9,49145,0.752
11 Austria,75.1,15.0,3.3,26911,0.56
12 Azerbaijan,71.9,17.8,5.7,32026,0.628
13 Bahamas,73.6,8.8,8.9,70141,0.729
14 Bahrain,66.9,17.5,7.6,79571,0.811
15 Bangladesh,84.6,8.9,3.6,72755,0.708
16 Barbados,57.7,19.9,9.4,61217,0.758
17 Belarus,70.1,18.0,12.8,35976,0.733
18 Belgium,69.8,16.8,12.5,21528,0.662
19 Belize,72.2,14.3,12.7,73697,0.844
20 Benin,58.9,11.5,3.3,35402,0.5
21 Bhutan,80.6,14.3,12.7,75328,0.879
22 Bolivia,66.5,11.7,13.1,6680,0.551
23 Bosnia and Herzegovina,73.5,19.4,13.0,17616,0.693
24 Botswana,57.4,13.2,12.9,63210,0.744
25 Brazil,68.2,8.1,3.2,44373,0.531
26 Brunei,57.9,8.3,12.6,56800,0.668
27 Bulgaria,65.6,19.9,10.2,55845,0.776
28 Burkina Faso,70.8,8.0,14.8,24604,0.619
29 Burundi,70.4,15.8,8.7,30553,0.637
30 Cabo Verde,84.6,16.0,10.0,67890,0.847
31 Cambodia,62.0,16.2,4.1,12093,0.481
32 Cameroon,70.1,9.8,6.8,11498,0.479
33 Canada,63.7,14.6,5.2,66562,0.69
34 Central African Republic,57.1,16.7,9.9,49509,0.686
35 Chad,59.5,15.6,3.0,30437,0.52
36 Chile,72.6,15.8,12.4,38603,0.724
37 China,77.3,17.5,5.6,16591,0.585
38 Colombia,81.6,17.2,8.8,45106,0.746
39 Comoros,66.8,11.9,4.2,5672,0.432
40 Congo,67.3,15.8,13.6,5858,0.599
41 Costa Rica,67.9,9.2,4.7,63775,0.633
42 Cote d'Ivoire,75.9,9.8,11.1,58352,0.733
43 Croatia,55.3,10.9,7.1,62208,0.632
44 Cuba,63.2,13.8,4.3,41410,0.574
45 Cyprus,68.0,9.5,4.7,66670,0.648
```

Activity 3.2: Understand the Dataset

After loading the dataset, its structure is explored to understand the available features, target variable, number of records, data types, and statistical properties.

The following aspects of the dataset are examined:

- Number of rows and columns
- Column names
- Data types
- Missing values
- Statistical summary
- Feature distributions
- Target variable

The four main independent variables are Life Expectancy, Mean Years of Schooling, Expected Years of Schooling, and GNI per Capita.

The dependent variable is the HDI Score.

Understanding the dataset helps identify data quality issues and ensures that the correct variables are selected for model development.

dataset is examined carefully to identify incomplete records.

Appropriate preprocessing techniques are applied wherever required to ensure that the dataset is clean and suitable for machine learning.

After

```
95 @app.route("/about")
96 def about():
97     return render_template("about.html")
98
99
100 if __name__ == "__main__":
101
102     # Open browser automatically (only once)
103     if environ.get("WERKZEUG_RUN_MAIN") == "true":
104         Timer(1, open_browser).start()
105
106     app.run(
107         debug=True,
108         host="0.0.0.0",
109         port=5001
110     )
```

preprocessing, the cleaned dataset is prepared for further model development.

Activity 4.3: Label Encoding

Machine learning algorithms require numerical data for model training. Therefore, categorical text values are converted into numerical representations wherever required.

The primary prediction target of the HDI Predictor is the numerical HDI Score. The predicted score is later interpreted into one of four human development categories:

- Very High Human Development
- High Human Development
- Medium Human Development
- Low Human Development

This classification makes the prediction easier for users to understand.

Activity 4.4: Prepare the Processed Dataset

After selecting the required variables, checking missing values, and processing the data, the dataset is prepared for machine learning.

The preprocessing stage ensures that the input data is clean, consistent, and suitable for model

training.

Feature scaling is applied where required to maintain consistent numerical ranges. The processed dataset is then used for train-test splitting and model development.

Milestone 5: Dividing the Dataset into Train and Test Data

Activity 5.1: Split the Dataset

In this milestone, the processed dataset is divided into training and testing sets.

The training dataset is used to train the machine learning model and help it learn the relationship between the four input indicators and the HDI score.

The testing dataset is used to evaluate the model using data that was not used during training.

The dataset is divided as follows:

- 80% of the data is used for training.
- 20% of the data is used for testing.

A random state is used to ensure consistent and reproducible results.

Dividing the dataset into training and testing sets provides a reliable method for evaluating the performance of the machine learning model on unseen data.

HDI_Predictor > tests > test_app.py > ...

```
1 import os
2 import unittest
3 import sqlite3
4 import numpy as np
5 from app.predict import predict_hdi, classify_hdi_category
6 from app import utils
7
8 class TestHDIPredictor(unittest.TestCase):
9     def setUp(self):
10         # Con (variable) original_db: Literal['database.db']
11         self.original_db = utils.DATABASE_PATH
12         utils.DATABASE_PATH = "test_database.db"
13         utils.init_db()
14
15     def tearDown(self):
16         # Restore the original database configuration and clean up
17         utils.DATABASE_PATH = self.original_db
18         if os.path.exists("test_database.db"):
19             os.remove("test_database.db")
20
21     def test_database_initialization(self):
22         """Test database and tables creation."""
23         conn = utils.get_db_connection()
24         cursor = conn.cursor()
25
```

Milestone 6: Fitting the Model

Activity 6.1: Train the Linear Regression Model

In this activity, the Linear Regression algorithm is used to train the machine learning model. Linear Regression is selected because the target variable, HDI Score, is a continuous numerical value.

The model learns the relationship between the following input indicators:

- Life Expectancy
- Mean Years of Schooling
- Expected Years of Schooling
- GNI per Capita

The trained model uses these relationships to estimate the HDI score for new user inputs.

```

predict.py 3 test_app.py 1 X Home - Human Development Index... requirements.txt .gitignore
HDI_Predictor > tests > test_app.py > ...
8 class TestHDIPredictor(unittest.TestCase):
9     def setUp(self):
10         # Configure a temporary database path for testing
11         self.original_db = utils.DATABASE_PATH
12         utils.DATABASE_PATH = "test_database.db"
13         utils.init_db()
14
15     def tearDown(self):
16         # Restore the original database configuration and clean up
17         utils.DATABASE_PATH = self.original_db
18         if os.path.exists("test_database.db"):
19             os.remove("test_database.db")
20
21     def test_database_initialization(self):
22         """Test database and tables creation."""
23         conn = utils.get_db_connection()
24         cursor = conn.cursor()
25
26         # Check users table
27         cursor.execute("SELECT name FROM sqlite_master WHERE type='table' AND name='users'")
28         self.assertIsNotNone(cursor.fetchone(), "Users table should exist.")
29
30         # Check predictions table
31         cursor.execute("SELECT name FROM sqlite_master WHERE type='table' AND name='predictions'")
32         self.assertIsNotNone(cursor.fetchone(), "Predictions table should exist.")
33

```

Activity 6.2: Generate Predictions

After training the Linear Regression model, predictions are generated using the testing dataset. The model receives the input features from the test data and generates predicted HDI scores. The predicted values are compared with the actual HDI values to understand how accurately the model performs.

This process helps verify whether the trained model can generate reliable predictions for unseen data.

```

HDI_Predictor > model > create_notebook.py > ...
1 import os
2 import json
3
4 # Setup notebooks directory
5 os.makedirs("notebooks", exist_ok=True)
6 notebook_path = os.path.join("notebooks", "HDI_Analysis.ipynb")
7
8 # Define cells structure
9 cells = [
10     {
11         "cell_type": "markdown",
12         "metadata": {},
13         "source": [
14             "# Human Development Index (HDI) Exploratory Data Analysis & Modeling\n",
15             "This notebook provides a comprehensive academic analysis of the Human Development Index (HDI) dataset,\n",
16         ]
17     },
18     {
19         "cell_type": "markdown",
20         "metadata": {},
21         "source": [
22             "## 1. Project Background and Indicators\n",
23             "The Human Development Index (HDI) is a statistical measure compiled by the United Nations to evaluate t\n",
24             "- **Health**: Measured by Life Expectancy at Birth.\n",
25             "- **Education**: Measured by Mean Years of Schooling and Expected Years of Schooling.\n",
26             "- **Standard of Living**: Measured by Gross National Income (GNI) per Capita."

```

Activity 6.3: Evaluate Model Performance

The performance of the trained Linear Regression model is evaluated using regression evaluation metrics.

The following metrics are used:

- Mean Absolute Error (MAE): Measures the average absolute difference between actual and

predicted values.

- Mean Squared Error (MSE): Measures the average squared difference between actual and predicted values.
- Root Mean Squared Error (RMSE): Represents the prediction error in the same unit as the target variable.
- R^2 Score: Measures how well the model explains the variation in the HDI score.

Milestone 7: Saving the Model

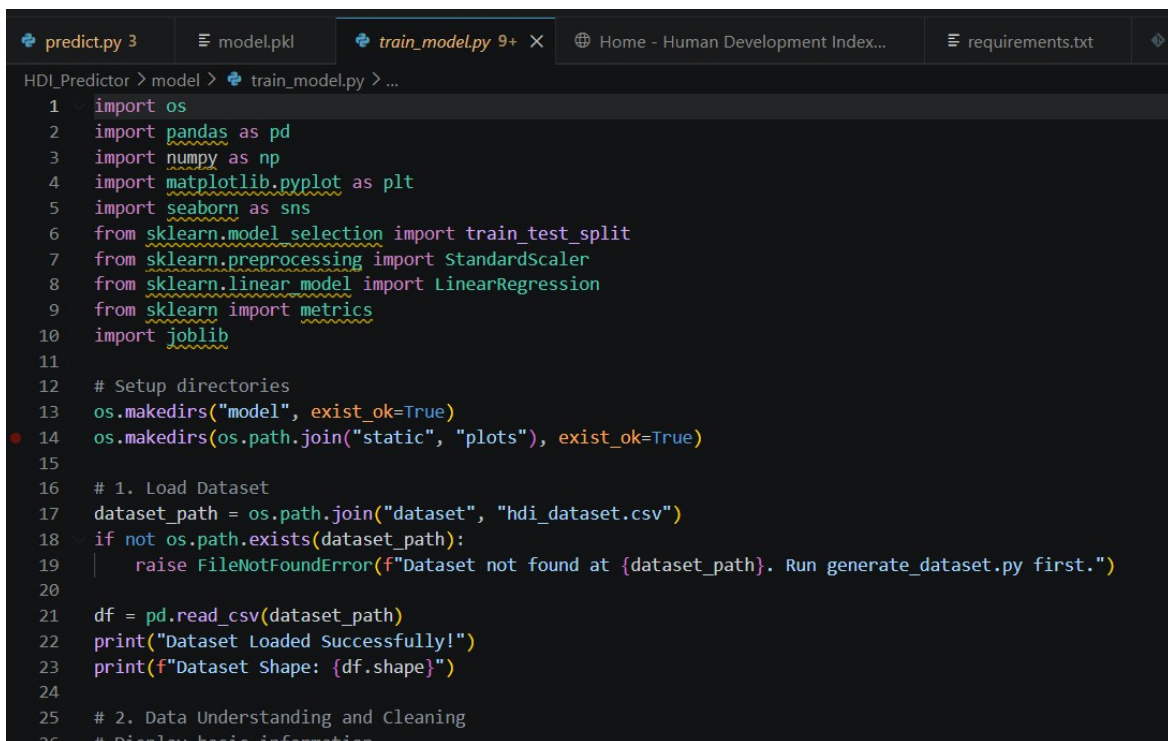
Activity 7.1: Save the Trained Model

After successful model training and evaluation, the trained Linear Regression model is saved using serialization.

Saving the model allows it to be reused by the Flask web application without training it again every time the application starts.

The trained model is stored inside the model folder of the project.

This improves application efficiency and separates the model training process from the prediction process.



```

predict.py 3  model.pkl  train_model.py 9+ X  Home - Human Development Index...  requirements.txt
HDI_Predictor > model > train_model.py > ...
1  import os
2  import pandas as pd
3  import numpy as np
4  import matplotlib.pyplot as plt
5  import seaborn as sns
6  from sklearn.model_selection import train_test_split
7  from sklearn.preprocessing import StandardScaler
8  from sklearn.linear_model import LinearRegression
9  from sklearn import metrics
10 import joblib
11
12 # Setup directories
13 os.makedirs("model", exist_ok=True)
14 os.makedirs(os.path.join("static", "plots"), exist_ok=True)
15
16 # 1. Load Dataset
17 dataset_path = os.path.join("dataset", "hdi_dataset.csv")
18 if not os.path.exists(dataset_path):
19     raise FileNotFoundError(f"Dataset not found at {dataset_path}. Run generate_dataset.py first.")
20
21 df = pd.read_csv(dataset_path)
22 print("Dataset Loaded Successfully!")
23 print(f"Dataset Shape: {df.shape}")
24
25 # 2. Data Understanding and Cleaning
26 # Display basic information
  
```

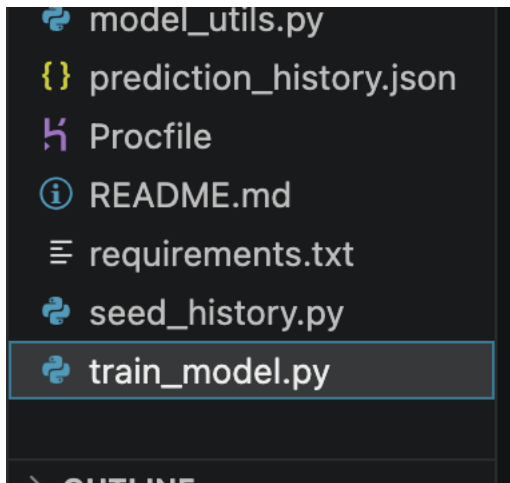
Activity 7.2: Store Model Files for Future Prediction

The required machine learning components are stored so that they can be loaded whenever a user requests an HDI prediction.

The saved components support:

- Fast prediction
- Model reuse
- Reduced application startup time
- Separation of model training and web application execution
- Consistent preprocessing of user inputs

The saved model files are used by the Flask backend to process new inputs and generate HDI predictions.



Milestone 8: Building the Flask Web Application

Activity 8.1: Develop the Flask Backend

In this activity, the Flask backend is developed to connect the web interface with the trained machine learning model. The Flask backend performs the following operations:

- Starts the web application
- Defines application routes
- Displays HTML pages
- Receives user input

- Validates input values
- Processes the input features
- Loads the trained machine learning model
- Generates the HDI score prediction
- Classifies the HDI category
- Displays the prediction result

The backend acts as the main connection between the user interface, machine learning model, and database.

```
HDI_Predictor > venv > Lib > site-packages > flask > app.py > ...
1  from __future__ import annotations
2
3  import collections.abc as cabc
4  import os
5  import sys
6  import typing as t
7  import weakref
8  from datetime import timedelta
9  from inspect import iscoroutinefunction
10 from itertools import chain
11 from types import TracebackType
12 from urllib.parse import quote as _url_quote
13
14 import click
15 from werkzeug.datastructures import Headers
16 from werkzeug.datastructures import ImmutableDict
17 from werkzeug.exceptions import BadRequestKeyError
18 from werkzeug.exceptions import HTTPException
19 from werkzeug.exceptions import InternalServerError
20 from werkzeug.routing import BuildError
21 from werkzeug.routing import MapAdapter
22 from werkzeug.routing import RequestRedirect
23 from werkzeug.routing import RoutingException
24 from werkzeug.routing import Rule
25 from werkzeug.serving import is_running_from_reloader
26 from werkzeug.wrappers import Response as BaseResponse
```

The user interface of the HDI Predictor is developed using HTML and CSS.

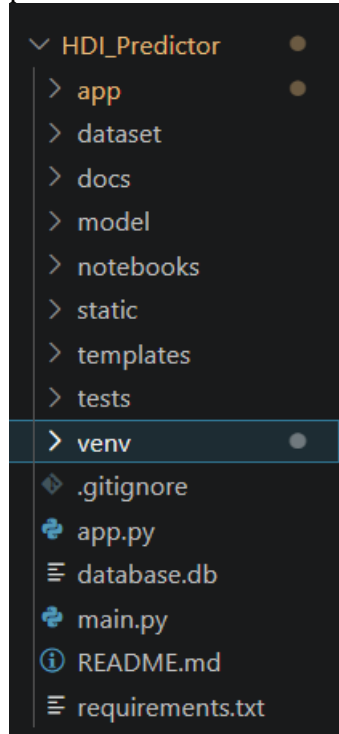
The templates folder contains the web pages required by the application.

The application includes:

- Home Page
- Predict HDI Page
- Prediction Result Page
- About Project Page

The static folder contains CSS files, images, visualization plots, and other resources required for the design of the application.

The HTML templates are integrated with Flask so that user requests can be processed and prediction results can be displayed dynamically.



Activity 8.3: Run, Test, and Validate the Application

After completing the backend and frontend integration, the complete Flask application is run and tested.

The application is started from the Visual Studio Code terminal.

The local web address displayed in the terminal is opened in a browser.

The application is tested by entering values for:

- Life Expectancy
- Mean Years of Schooling
- Expected Years of Schooling
- GNI per Capita

After entering the values, the user clicks the Predict button.

The Flask backend processes the inputs, sends them to the trained machine learning model, generates the HDI score, determines the human development category, and displays the result.

The complete application is validated to ensure accurate predictions and smooth functionality.

Exploring the Website's Web Pages

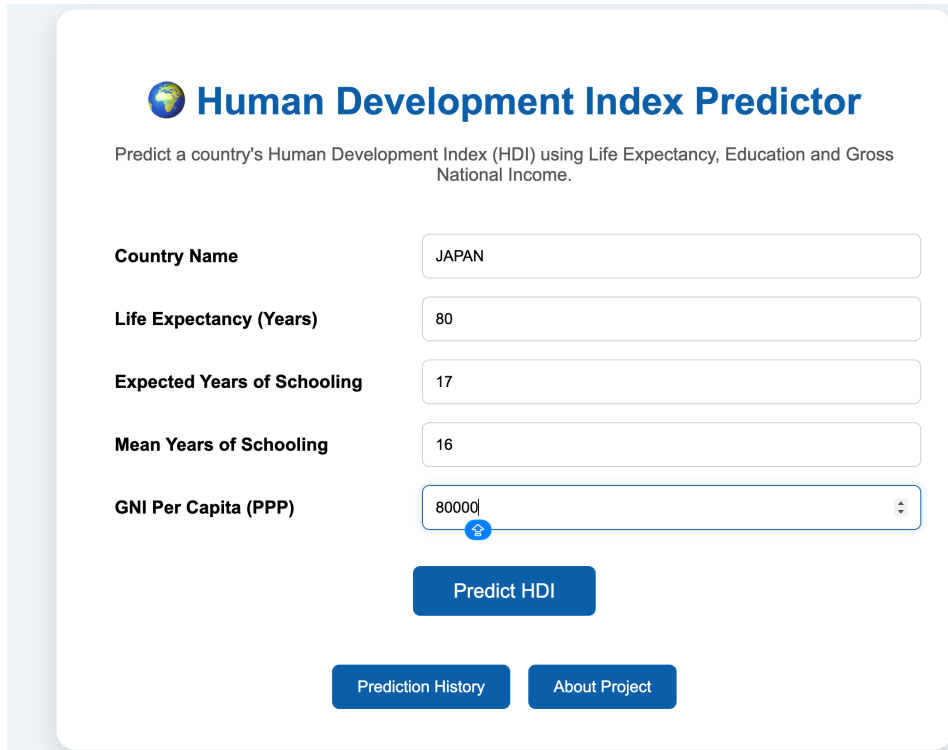
Home Page

The Home Page is the main landing page of the HDI Predictor. It introduces the concept of the Human Development Index and explains the purpose of the project.

The page highlights the four main indicators used by the prediction system:

- Life Expectancy
- Mean Years of Schooling
- Expected Years of Schooling
- GNI per Capita

The Home Page also provides an overview of the project and presents exploratory data analysis visualizations.



Human Development Index Predictor

Predict a country's Human Development Index (HDI) using Life Expectancy, Education and Gross National Income.

Country Name: JAPAN

Life Expectancy (Years): 80

Expected Years of Schooling: 17

Mean Years of Schooling: 16

GNI Per Capita (PPP): 80000

Predict HDI

Prediction History About Project

Four

The

**Dimension
Parameters**

Four
Dimension
Parameters

section explains the four major indicators used by the machine learning model.

- Life Expectancy: Represents the average number of years a newborn is expected to live under current mortality conditions. It reflects the overall health of the population.
- Mean Years of Schooling: Represents the average number of years of education received by people aged 25 years and older.
- Expected Years of Schooling: Represents the number of years of schooling that a child entering the education system can expect to receive.
- GNI per Capita: Represents Gross National Income per person and reflects the economic standard of living.

These four indicators collectively represent the major health, education, and living-standard dimensions of human development.

HDI Prediction Result

JAPAN

Indicator	Value
Life Expectancy	80.0 Years
Expected Years of Schooling	17.0 Years
Mean Years of Schooling	16.0 Years
GNI Per Capita (PPP)	\$80000.0

Predicted HDI Score

0.889

Development Category

Very High Human Development

Predict Another Country

Prediction History

About Project

Exploratory Data Analysis Preview

The Exploratory Data Analysis section presents visual insights obtained from the HDI dataset.

The visualizations help users understand:

- Relationships between development indicators
- Correlations among features
- Distribution of values
- Factors associated with higher or lower HDI scores

The section includes visualizations such as the Feature Correlation Heatmap and HDI Score relationship plots.

These visualizations make the data easier to understand and provide useful insights into the factors that influence human development.

Predict HDI Page

The Predict HDI Page provides an interactive form where users enter the four required human development indicators.

The input fields include:

- Life Expectancy
- Mean Years of Schooling
- Expected Years of Schooling
- GNI per Capita

After entering the required values, the user clicks the Predict button.

The application sends the values to the Flask backend, where they are validated, processed, and passed to the trained machine learning model.

Indicator	Entered Value	Dimension
Life Expectancy	78.5 Years	Health
Mean Years of Schooling	12.5 Years	Education
Expected Years of Schooling	15.6 Years	Education
Gross National Income (GNI) per Capita	\$45,000.00	Standard of Living

The Prediction Result Page displays the output generated by the trained machine learning model.

The page presents:

- Predicted HDI Score
- HDI Development Category

The predicted score is classified into one of the following categories:

- Very High Human Development
- High Human Development
- Medium Human Development
- Low Human Development

The result page provides a clear and user-friendly representation of the predicted human development level.

Predicted HDI Score

0.889

Development Category

Very High Human Development

About Project Page

The About Project Page provides information about the purpose, importance, workflow, and technologies used in the HDI Predictor.

The page explains how machine learning is used to analyze health, education, and income indicators to estimate the Human Development Index.

It also helps users understand the objective of the project and the role of the four development indicators in measuring overall human well-being.

About Human Development Index Predictor

The Human Development Index (HDI) is a statistical composite index used to measure a country's overall development. It considers three major indicators:

- Life Expectancy
- Education
- Gross National Income (GNI) Per Capita

This application predicts the HDI score using a Machine Learning model trained on historical HDI data.

Development Categories

HDI Score	Category
0.800 - 1.000	Very High Human Development
0.700 - 0.799	High Human Development
0.550 - 0.699	Medium Human Development
Below 0.550	Low Human Development

[Back to Home](#)

CONCLUSION:-

The **Human Development Index (HDI) Predictor** project successfully demonstrates the application of **Machine Learning** and **Web Development** to estimate a country's Human Development Index based on four key development indicators: **Life Expectancy**, **Expected Years of Schooling**, **Mean Years of Schooling**, and **Gross National Income (GNI) per Capita**. The system provides users with an intuitive web interface for entering country information and receiving an accurate HDI prediction along with the corresponding human development category.

The application is developed using the **Flask** framework for the backend and **HTML, CSS, and JavaScript** for the frontend, while the prediction model is trained using **Scikit-learn's Random Forest Regressor**. The trained model is integrated into the web application using **Joblib**, enabling real-time predictions. Prediction history is also maintained to allow users to review previous results, making the application more practical and user-friendly.

The project demonstrates how artificial intelligence and predictive analytics can support researchers, students, policymakers, and development organizations in understanding the relationship between socio-economic indicators and human development. The system reduces manual analysis, provides quick insights, and offers a scalable platform for future enhancements.

Overall, the **HDI Predictor** is a reliable, efficient, and practical decision-support application that combines machine learning with modern web technologies to provide meaningful human development predictions. Future improvements may include incorporating additional socio-economic indicators, integrating live datasets from international organizations, providing graphical analytics dashboards, and deploying the application on cloud platforms for global accessibility.

THANKYOU